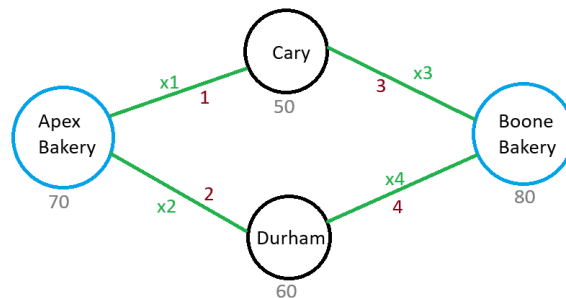


Reza Jahani	ECE572	Due Date: 12/1/2025
Rjahani@ncsu.edu	HW4	

1.



Based on the problem statement, let

x_1 : Shipped cakes from Apex to Cary

x_2 : Shipped cakes from Apex to Durham

x_3 : Shipped cakes from Boone to Cary

x_4 : Shipped cakes from Boone to Durham

Therefore, the function to minimize is derived.

$$f(x) = x_1 + 2x_2 + 3x_3 + 4x_4$$

And the constraints can be formulated as follows.

$$(I) \ x_1 + x_2 \leq 70$$

$$(II) \ x_3 + x_4 \leq 80$$

$$(III) \ x_1 + x_3 = 50$$

$$(IV) \ x_2 + x_4 = 60$$

The linear programming problem in a standard form is formulated below

$$\min \ x_1 + 2x_2 + 3x_3 + 4x_4$$

subject to

$$x_1 + x_2 + x_5 = 70$$

$$x_3 + x_4 + x_6 = 80$$

$$x_1 + x_3 = 50$$

$$x_2 + x_4 = 60$$

$$x_1, x_2, x_3, x_4, x_5, x_6 \geq 0$$

2.

To utilize the built-in function `<linprog()>`, we will employ the function in the format of $\text{linprog}(f, A, b, A_{eq}, b_{eq})$ to incorporate the inequalities and equalities simultaneously.

To this end, we consider the optimization problem we formulated in the beginning.

$$\begin{aligned} \min \quad & [1 \ 2 \ 3 \ 4] \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \\ \text{subject to} \quad & \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \leq \begin{bmatrix} 70 \\ 80 \end{bmatrix} \\ & \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 50 \\ 60 \end{bmatrix} \end{aligned}$$

Solving the linear programming problem,

```
>> x_star
x_star =
    50
    20
     0
    40

>> f_val
f_val =
    250

fx >>
```

$$x_1 = 50, x_2 = 20, x_3 = 0, x_4 = 40$$

- Solving it with format we formulated previously in Q1 also results in the same optimal point. (Available on the script attached.)

3.

The optimization problem can be formulated as follows:

$$\begin{aligned} & \min \sum_{j=1}^n p_j \\ & \text{subject to} \\ & \forall 1 \leq i \leq m; \sum_{j=1}^n g_{ij} p_j \geq P \\ & \forall 1 \leq j \leq n; p_j \geq 0 \end{aligned}$$

To further simplify and transform the problem into a linear programming format, we adjust the optimization problem with a few algebraic manipulations.

$$\begin{aligned} & \min \sum_{j=1}^n p_{ij} \\ & \text{subject to} \\ & \forall 1 \leq i \leq m; \sum_{j=1}^n g_{ij} p_{ij} - s_i = P \\ & \forall i; s_i \geq 0, \forall j; p_j \geq 0 \end{aligned}$$

Transforming it into matrix format:

$$\begin{aligned} & \min \mathbf{1}^T p \\ & \text{subject to} \\ & \mathbf{G}^T p - s = P \cdot \mathbf{1} \\ & p \geq 0, s \geq 0 \end{aligned}$$

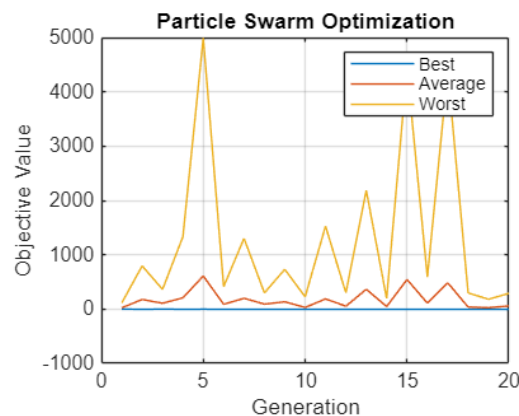
Where, $s_{m \times 1}$ represents the slacks vector, $p = [p_j]_{n \times 1}$, and $\mathbf{G} = [g_{ji}]_{n \times m}$.

4.

Implementing the Particle Swarm Optimization following the same steps as instructed through the slides, we minimize the discussed function.

```
Gen 1: f* = 3.485241 at [-0.2392, -0.4256]
Gen 2: f* = 3.485241 at [0.3392, -0.7849]
Gen 3: f* = 3.485241 at [0.3392, -0.7849]
Gen 4: f* = 3.485241 at [0.3392, -0.7849]
Gen 5: f* = 3.485241 at [0.3392, -0.7849]
Gen 6: f* = 3.485241 at [0.3392, -0.7849]
Gen 7: f* = 3.485241 at [0.5900, -0.6731]
Gen 8: f* = 3.485241 at [0.5900, -0.6731]
Gen 9: f* = 3.485241 at [0.5900, -0.6731]
Gen 10: f* = 3.485241 at [0.5900, -0.6731]
Gen 11: f* = 3.485241 at [0.5900, -0.6731]
Gen 12: f* = 3.485241 at [0.5900, -0.6731]
Gen 13: f* = 3.485241 at [0.5900, -0.6731]
Gen 14: f* = 3.485241 at [0.6332, -0.6457]
Gen 15: f* = 3.485241 at [0.6332, -0.6457]
Gen 16: f* = 3.485241 at [0.6332, -0.6457]
Gen 17: f* = 3.485241 at [0.6332, -0.6457]
Gen 18: f* = 3.485241 at [0.6332, -0.6457]
Gen 19: f* = 3.485241 at [0.6332, -0.6457]
Gen 20: f* = 3.485241 at [0.6332, -0.6457]
```

$x1 = 0.6332$, $x2 = -0.6457$, $f(x1, x2) = 3.485241$



- Code for this part is available in the attached script.
- A couple steps are added to the algorithm for ensuring the convergence and prevention of algorithm malfunctioning, including:
 - Initialization of the first generation within a limit based on an upper and lower bound.
 - Clipping the new generation to fit them within the boundaries.
 - Other than that, every step is the same as instructed algorithms on the slides.

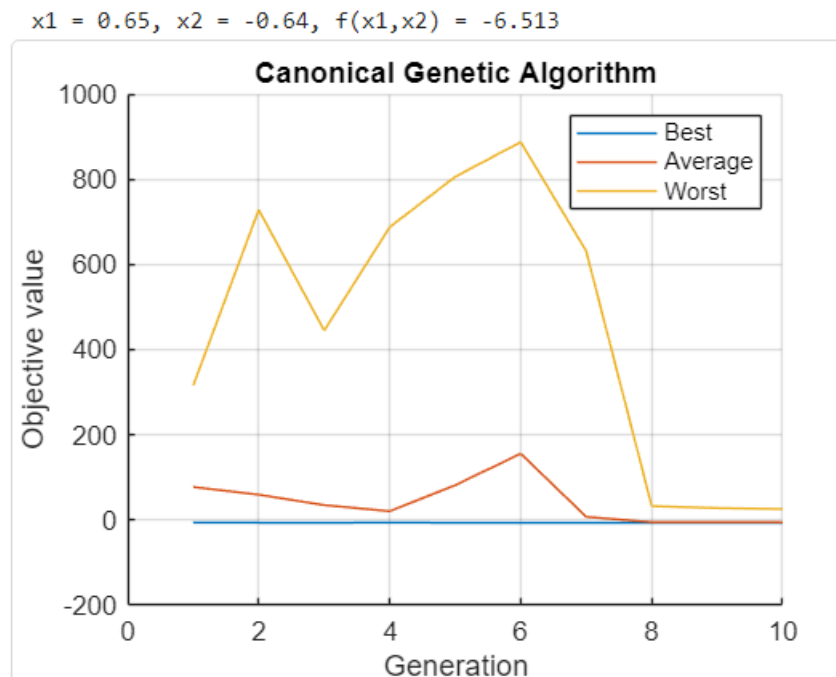
Canonical Genetic Algorithm.

This algorithm operates based on the following steps.

- (i) Choosing a random population.
- (ii) Evaluation of the population based on fitness function and creating a mating pool from the top performing particles.
- (iii) Applying cross-over and mutation
- (iv) Repeating the algorithm for a couple generations.

- The code for this implementation is available in the attached script.

As requested, the worst, average, and best performance of populations over the generations are plotted.



- One slight difference that my implementation has is in forming the mating pool. In this algorithm, we pick the top performing particles of the population instead of applying other methods like Roulette-Wheel selection. This reduces computational complexity while preserving the desired performance.

5.

The primal problem as mentioned in the instructions is as follows.

$$\begin{aligned}
 &\text{maximize} && 10I_2 + 6I_4 + 20I_5, \\
 &\text{subject to} && I_1 = I_2 + I_3, \\
 &&& I_3 = I_4 + I_5, \\
 &&& I_1 \leq 4, \quad I_2 \leq 3, \quad I_3 \leq 3, \quad I_4 \leq 2, \quad I_5 \leq 2, \\
 &&& I_1, I_2, I_3, I_4, I_5 \geq 0.
 \end{aligned}$$

For forming the **Dual Problem**, we need all constraints to follow the notation of $Ax \leq b$. To this end, we convert the notation of first two constraint, further integrating it with other constraint, we derive the following.

$$\begin{aligned}
 &I_1 - I_2 - I_3 \leq 0 \\
 &-I_1 + I_2 + I_3 \leq 0 \\
 &I_3 - I_4 - I_5 \leq 0 \\
 &-I_3 + I_4 + I_5 \leq 0 \\
 &I_1 \leq 4, I_2 \leq 3, I_3 \leq 3, I_4 \leq 2, I_5 \leq 2
 \end{aligned}$$

$$x = [I_1 \ I_2 \ I_3 \ I_4 \ I_5]^T \quad c = [0 \ 10 \ 0 \ 6 \ 20]^T$$

$$A = \begin{bmatrix} 1 & -1 & -1 & 0 & 0 \\ -1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1 & -1 \\ 0 & 0 & -1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad b = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 4 \\ 3 \\ 3 \\ 2 \\ 2 \end{bmatrix}$$

Each constraint is translated as a variable in the Dual Problem.

$$f(y) = b^T y = 4y_5 + 3y_6 + 3y_7 + 2y_8 + 2y_9$$

Similarly, the constraint of the Dual Problem is formed based on the primal problem variables.

$$A^T y \geq c$$

$$\begin{aligned}
 &y_1 - y_2 + y_5 \geq 0 \\
 &-y_1 + y_2 + y_6 \geq 10 \\
 &-y_1 + y_2 + y_3 - y_4 + y_7 \geq 0 \\
 &-y_3 + y_4 + y_8 \geq 6 \\
 &-y_3 + y_4 + y_9 \geq 20
 \end{aligned}$$

Therefore, the Dual Problem is formulated as follows.

$$\begin{aligned}
 &\min 4y_5 + 3y_6 + 3y_7 + 2y_8 + 2y_9 \\
 &\text{subject to} \\
 &y_1 - y_2 + y_5 \geq 0 \\
 &-y_1 + y_2 + y_6 \geq 10 \\
 &-y_1 + y_2 + y_3 - y_4 + y_7 \geq 0 \\
 &-y_3 + y_4 + y_8 \geq 6 \\
 &-y_3 + y_4 + y_9 \geq 20 \\
 &\forall 1 \leq i \leq 9; y_i \geq 0
 \end{aligned}$$

We first solve this problem using `linprog()` function and then proceed with finding the optimum value for the primal problem. We just need to multiply the constraints by -1 to enable the `linprog()` function to utilize this problem.

After solving the Dual Problem using `linprog()` and checking which of the 5 constraints are set to equality, we realize that.

```

eq_c =
5x1 logical array
    1
    1
    1
    0
    1

y_opt =
    0
   10
    0
   10
   10
    0
    0
    0
   10

```

This illustrates that the value for I_4^* should be 0. Since the constraints 1 to 3 and 5 are set to equality based on the dual problem solution, the corresponding linear equations hold.

$$\begin{aligned}
 -I_1 + I_2 + I_3 &= 0 \\
 -I_3 + I_4 + I_5 &= 0 \\
 I_1 &= 4 \\
 I_5 &= 2
 \end{aligned}$$

The system of linear equations above determines the final optimum answer for the primal problem.

$$\begin{aligned}
 I_1^* &= 4, I_2^* = 2, I_3^* = 2, I_4^* = 0, I_5^* = 2 \\
 f_{\text{primal}} &= 10I_2^* + 6I_4^* + 20I_5^* = 6
 \end{aligned}$$

- This problem is implemented in the script attached.